

Linformer-IDS: A Lightweight Transformer Architecture for Efficient Intrusion Detection in IoT and Industrial IoT Networks

Anonymous

Abstract—The proliferation of Internet-of-Things (IoT) and Industrial IoT (IIoT) devices has amplified the attack surface for modern networks, creating an urgent need for intrusion detection systems that balance detection performance with resource efficiency. Traditional transformer-based models achieve high accuracy but suffer from quadratic time and memory complexity with respect to sequence length, making them impractical for resource-constrained edge environments. This work proposes Linformer-IDS, a lightweight transformer-based intrusion detection framework that leverages low-rank projections to approximate self-attention, reducing computational complexity while maintaining competitive detection performance.

We design an end-to-end preprocessing and training pipeline tailored for IoT/IIoT network telemetry and evaluate the system on multiple benchmark datasets, including CIC-IoT and Edge-IIoT set variants, under both binary and multi-class classification scenarios. The Linformer-IDS architecture incorporates dataset-specific preprocessing, a reproducible training regime, and a unified evaluation toolkit that produces comprehensive per-dataset metrics and comparison reports.

I. INTRODUCTION

The exponential proliferation of Internet of Things (IoT) and Industrial IoT (IIoT) devices has fundamentally transformed modern network architectures, with over 30 billion connected devices projected by 2025 [?]. This unprecedented growth has enabled revolutionary applications across smart cities, industrial automation, healthcare monitoring, and critical infrastructure management. However, the massive scale and heterogeneous nature of IoT deployments have simultaneously created an expanded attack surface that traditional security mechanisms struggle to protect [1]. IoT devices, often characterized by limited computational resources, minimal security features, and diverse communication protocols, present unique vulnerabilities that sophisticated adversaries increasingly exploit through targeted attacks ranging from distributed denial-of-service (DDoS) campaigns to advanced persistent threats (APTs).

Intrusion Detection Systems (IDS) have emerged as a critical defensive layer for IoT ecosystems, providing continuous monitoring and anomaly detection capabilities essential for maintaining network integrity. Traditional signature-based IDS approaches, while effective against known threats, fail to detect zero-day exploits and polymorphic malware variants prevalent in modern IoT attack landscapes [2]. Machine learning-based IDS solutions offer superior adaptability and generalization capabilities, particularly deep learning models that can automatically extract complex features from

high-dimensional network traffic data. Recent advances in transformer architectures have demonstrated state-of-the-art performance across numerous sequence modeling tasks, suggesting significant potential for network intrusion detection where temporal dependencies and long-range correlations are critical for identifying sophisticated multi-stage attacks.

Despite their impressive capabilities, transformer-based models present fundamental deployment challenges in resource-constrained IoT environments. The self-attention mechanism, central to transformer architectures, exhibits quadratic computational complexity $\mathcal{O}(n^2d)$ with respect to sequence length n and embedding dimension d , rendering real-time inference infeasible on edge devices with limited processing power and memory [3]. IoT edge nodes typically operate with 1-2 TFLOPS of computational capacity, 256MB-2GB of RAM, and strict power consumption constraints, specifications incompatible with standard transformer models requiring gigaflops of computation per inference. Furthermore, the heterogeneous nature of IoT deployments demands models capable of processing diverse traffic patterns across multiple protocols including MQTT, CoAP, Modbus, and proprietary industrial communication standards, necessitating architectural flexibility absent in monolithic transformer designs.

The critical gap between transformer model capabilities and IoT deployment constraints motivates the development of lightweight, efficient architectures that preserve detection accuracy while dramatically reducing computational overhead. Linear attention mechanisms have emerged as a promising solution, approximating full self-attention through low-rank matrix factorization techniques that reduce complexity to $\mathcal{O}(nkd)$ where $k \ll n$ represents the projection dimension [?]. This linearization enables deployment on resource-constrained devices while maintaining the transformer’s ability to model long-range dependencies crucial for detecting distributed attack patterns. However, adapting linear attention for intrusion detection requires careful consideration of domain-specific characteristics including temporal locality of attack signatures, protocol-specific feature representations, and real-time processing requirements.

This paper presents Linformer-IDS, a novel lightweight transformer architecture specifically optimized for efficient intrusion detection in IoT and IIoT networks. Our approach leverages Linformer’s linear self-attention mechanism combined with sparse windowed attention to exploit the temporal

locality inherent in network attack patterns, achieving computational complexity of $\mathcal{O}(nwk d)$ where w represents the attention window size. The architecture incorporates several key innovations tailored for IoT security applications. First, we introduce a compact encoder design with reduced hidden dimensions ($d_{model} = 128$) and fewer layers ($N = 4$) that maintains sufficient representational capacity for intrusion detection while reducing model size by 5-10 $\times$ compared to standard transformers. Second, we develop a hybrid attention mechanism combining low-rank approximation with local windowing that focuses computation on temporally proximate events where attack correlations are strongest. Third, we implement an end-to-end preprocessing pipeline supporting multiple IoT protocols and handling the extreme class imbalance characteristic of intrusion detection datasets where malicious traffic typically represents less than 1% of network flows.

Our primary contributions are threefold. First, we design and implement a resource-aware transformer architecture achieving linear computational complexity while preserving competitive detection performance across diverse attack scenarios. The Linformer-IDS model requires only 2% of the computational operations needed by standard transformer attention for typical IoT deployment parameters, enabling sub-10ms inference latency on edge devices. Second, we develop a reproducible and modular training pipeline incorporating dataset-specific preprocessing, automated hyperparameter optimization, and cross-dataset evaluation capabilities that facilitate systematic comparison across multiple IDS benchmarks. Third, we conduct comprehensive empirical validation on two large-scale IoT security datasets—Edge-IIoTset and CICIOT2023—demonstrating that Linformer-IDS achieves precision, recall, and F1-scores exceeding 95% for most attack categories while maintaining model sizes under 10MB suitable for edge deployment.

The remainder of this paper is organized as follows. Section II reviews related work in IoT intrusion detection and efficient transformer architectures. Section III presents the Linformer-IDS architecture and theoretical foundations. Section IV details our training methodology and experimental setup. Section V analyzes experimental results and performance comparisons. Section VI discusses deployment considerations and limitations. Finally, Section VII concludes with future research directions.

II. RELATED WORKS

The evolution of intrusion detection systems for IoT environments has progressed through multiple paradigms, from traditional signature-based approaches to sophisticated deep learning architectures. This section critically examines existing research across three key areas: conventional IDS methodologies for IoT networks, transformer-based approaches for network security, and efficient attention mechanisms addressing computational constraints.

A. Traditional Machine Learning Approaches for IoT Intrusion Detection

Early IoT intrusion detection systems relied primarily on statistical and shallow machine learning methods to identify anomalous network behavior. Support Vector Machines (SVMs) have been extensively applied for IoT security, with [?] proposing SVELTE, a lightweight IDS achieving 90% detection accuracy on 6LoWPAN networks while maintaining minimal memory footprint. However, SVMs struggle with non-linear attack patterns and require manual feature engineering, limiting their effectiveness against sophisticated multi-stage attacks. Random Forest classifiers demonstrated improved performance, as shown by [?], achieving 99% accuracy on IoT DDoS detection tasks through ensemble voting mechanisms. Nevertheless, these methods exhibit degraded performance when confronting previously unseen attack variants and cannot effectively model temporal dependencies critical for detecting distributed attacks spanning multiple time windows.

Decision tree-based approaches offered interpretability advantages crucial for security analysis. [?] developed a three-layer IDS using C4.5 decision trees, achieving real-time classification with 95% accuracy on smart home traffic. The hierarchical structure enabled protocol-specific detection rules, improving performance on heterogeneous IoT networks. However, decision trees suffer from overfitting on imbalanced datasets where malicious traffic represents less than 1% of samples, a fundamental characteristic of production IoT environments. K-Nearest Neighbors (KNN) classifiers provided instance-based learning capabilities, with [?] achieving 84% detection rates on IoT botnet traffic. Despite reasonable accuracy, KNN's computational complexity during inference makes it impractical for resource-constrained edge devices requiring sub-millisecond response times.

Hybrid approaches combining multiple shallow learners attempted to address individual method limitations. [?] proposed RPiDS, integrating Naive Bayes, decision trees, and neural networks on Raspberry Pi devices, demonstrating feasibility of edge-based detection. While achieving 92% overall accuracy, the system required extensive preprocessing and could not adapt to evolving attack patterns without complete retraining. These traditional methods, though computationally efficient, fundamentally lack the representational capacity to model complex attack behaviors emerging in modern IoT ecosystems.

B. Deep Learning Evolution in Network Intrusion Detection

The adoption of deep learning revolutionized network intrusion detection by enabling automatic feature extraction from raw traffic data. Convolutional Neural Networks (CNNs) emerged as powerful feature extractors for spatial patterns in network flows. [?] demonstrated CNN-based IDS achieving 95% accuracy on NSL-KDD dataset, automatically learning discriminative features from packet payloads. [?] extended this approach with multi-channel CNNs processing different protocol layers independently, achieving 97%

detection rates on IoT-specific attacks. However, CNNs' local receptive fields limit their ability to capture long-range temporal dependencies essential for identifying coordinated attacks distributed across extended time periods.

Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks addressed temporal modeling requirements inherent in sequential network traffic. [?] proposed a multi-layer RNN-IDS achieving 99.81% accuracy on binary classification and 99.52% on multi-class intrusion detection. The recurrent architecture effectively captured attack progression patterns, particularly for multi-stage exploits. [?] implemented bidirectional LSTMs for IoT anomaly detection, leveraging forward and backward temporal context to identify subtle deviations from normal behavior. Despite impressive accuracy, LSTM-based approaches suffer from vanishing gradient problems for long sequences and cannot parallelize training, resulting in prohibitive computational costs for large-scale IoT deployments.

Autoencoder-based anomaly detection provided unsupervised learning capabilities crucial for zero-day attack detection. [?] developed N-BaIoT using deep autoencoders to profile normal IoT device behavior, detecting Mirai botnet variants with 99% precision without labeled attack data. [?] proposed Kitsune, an ensemble of autoencoders achieving real-time anomaly detection with false positive rates below 0.1%. While effective for deviation detection, autoencoders struggle with targeted attacks designed to mimic legitimate traffic patterns and require continuous retraining to adapt to evolving network behaviors.

Graph Neural Networks (GNNs) recently emerged for modeling network topology and communication patterns. [?] applied GraphSAGE to IoT intrusion detection, representing devices as nodes and communications as edges, achieving 98% accuracy while capturing network-wide attack propagation. However, GNNs' computational requirements for message passing and aggregation operations exceed typical IoT device capabilities, limiting deployment to centralized architectures.

C. Transformer Architectures in Network Security

The transformer architecture's success in natural language processing motivated adaptation for network security applications. [3] introduced the original transformer model, demonstrating superior sequence modeling through self-attention mechanisms eliminating recurrence requirements. The architecture's ability to capture long-range dependencies and parallelize computation suggested significant potential for network traffic analysis where attack patterns span extensive temporal windows.

[?] pioneered transformer application to intrusion detection with RTIDS, achieving 99.5% accuracy on UNSW-NB15 dataset through multi-head attention mechanisms modeling complex feature interactions. The model demonstrated particular effectiveness on sophisticated attacks involving multiple protocols and extended reconnaissance phases. [?] developed a hierarchical transformer architecture processing

packet-level and flow-level features independently, achieving state-of-the-art performance on CIC-IDS2017 benchmark with 99.8% precision. The dual-level attention enabled fine-grained attack signature detection while maintaining computational efficiency through feature dimensionality reduction.

Vision Transformer (ViT) adaptations treated network traffic as image-like spatial-temporal patterns. [?] converted flow features into 2D representations, applying ViT for visual pattern recognition achieving 98.7% accuracy on IoT botnet detection. This approach leveraged pre-trained vision models, reducing training requirements for limited IoT security datasets. However, the image conversion process introduced additional computational overhead unsuitable for real-time edge processing.

Despite impressive accuracy, standard transformer implementations remain computationally prohibitive for IoT deployment. [?] analyzed transformer complexity for network security, demonstrating that achieving real-time performance on edge devices requires 100× complexity reduction while maintaining detection accuracy. This fundamental limitation motivates research into efficient attention mechanisms preserving transformer capabilities within IoT resource constraints.

D. Efficient Attention Mechanisms and Model Compression

The quadratic complexity of self-attention has driven extensive research into efficient approximations maintaining model expressiveness. [?] introduced Linformer, approximating self-attention through low-rank matrix projections reducing complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$. Theoretical analysis proved that attention matrices exhibit low-rank structure, enabling accurate approximation with projection dimension $k \ll n$. Empirical validation on language modeling tasks demonstrated comparable performance with 50× speed improvement, suggesting applicability to sequence-based intrusion detection.

[?] proposed Reformer using locality-sensitive hashing (LSH) to reduce attention complexity to $\mathcal{O}(n \log n)$. The approach groups similar tokens through hashing functions, computing attention only within buckets. While effective for natural language, LSH's assumption of similarity-based attention patterns may not hold for network traffic where attacks deliberately mimic benign behavior. [?] developed Performers using random feature approximations of Gaussian kernels, achieving linear complexity through kernel trick applications. The method demonstrated strong theoretical guarantees but required careful feature map design for domain-specific applications.

Sparse attention patterns offered an alternative efficiency approach. [?] introduced Sparse Transformers with fixed attention patterns reducing computation to $\mathcal{O}(n\sqrt{n})$. The method combined local windowed attention with strided patterns capturing long-range dependencies. [?] extended this concept with Longformer's sliding window attention, demonstrating effectiveness on long documents. For network security, [?] proposed BigBird combining random, window,

and global attention patterns, achieving linear complexity while maintaining expressiveness.

Model compression techniques complemented architectural efficiency improvements. [?] demonstrated knowledge distillation reducing model size by 40% while retaining 97% of performance. Quantization approaches converted 32-bit floating-point weights to 8-bit integers, achieving 4× compression with minimal accuracy loss. [?] combined pruning, quantization, and Huffman coding for 35× compression ratios. However, aggressive compression often degraded performance on minority attack classes crucial for security applications.

E. Research Gaps and Motivation

Existing research reveals critical gaps in developing practical IDS solutions for resource-constrained IoT environments. Traditional machine learning methods lack representational capacity for sophisticated attacks, while deep learning approaches impose prohibitive computational requirements. Standard transformers achieve state-of-the-art detection accuracy but cannot deploy on edge devices with limited processing capabilities. Efficient attention mechanisms developed for NLP tasks have not been systematically adapted for network intrusion detection's unique characteristics including temporal locality, extreme class imbalance, and real-time processing requirements.

Furthermore, current IoT-specific IDS research lacks comprehensive evaluation across diverse datasets and deployment scenarios. Most studies optimize for accuracy metrics without considering inference latency, memory footprint, or power consumption critical for edge deployment. The absence of reproducible preprocessing pipelines and standardized evaluation protocols hinders systematic comparison and practical adoption. These limitations motivate our development of Linformer-IDS, specifically designed to bridge the gap between detection performance and deployment feasibility through domain-optimized linear attention mechanisms and comprehensive empirical validation.

III. PROPOSED APPROACH

This section presents a analysis of the Linformer architecture and its application to intrusion detection in IoT environments. First, we establish the computational limitations of standard Transformers for constraint devices of IoT devices, derive the Linformer attention framework, and demonstrate how linear complexity is achieved while maintaining competitive accuracy for intrusion detection.

A. Motivation

Standard Transformer-based attention mechanisms exhibit quadratic time complexity $\mathcal{O}(n^2d)$ with respect to sequence length n , where d is the embedding dimension. This prohibitive complexity renders real-time deployment infeasible on resource-constrained IoT devices. Our methodology addresses this challenge through rank-based approximation of attention matrices, enabling better intrusion detection without performance degradation.

B. Attention Mechanism Fundamentals

We begin by understanding & establishing computational and space complexity of standard Transformer attention proposed in the "Attention is all you need" [3], providing theoretical foundation for understanding of the Linformer and how we overcomes these limitations for resource constraint IoT devices.

1) *The Scaled Dot-Product Attention Mechanism:* The core computation in standard Transformer attention is the Scaled Dot-Product Attention, defined as:

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1)$$

where:

- **Query Matrix** ($Q \in \mathbb{R}^{n \times d_k}$): Contains n query vectors representing information the model seeks. Queries act as learnable filters optimized during training to extract task-relevant patterns.
- **Key Matrix** ($K \in \mathbb{R}^{n \times d_k}$): Contains n key vectors with matching dimensionality. Keys serve as indices enabling queries to determine relevance through dot product comparisons. The matrix product QK^T generates pairwise comparisons capturing long-range dependencies.
- **Value Matrix** ($V \in \mathbb{R}^{n \times d_v}$): Contains n value vectors of dimension d_v . After Softmax normalization, attention weights scale values to produce output representations. The separation of keys (for matching) and values (for content) enables independent learning of what to attend to and what information to extract.
- **Key/Query Dimension** (d_k): Typically $d_k = d_v = d/h$, where d is hidden dimension and h denotes attention heads. Multi-head decomposition enables joint attention across different representation subspaces, allowing each head to specialize in distinct attention patterns.
- **Scaling Factor** ($\sqrt{d_k}$): Normalizes dot products to maintain training stability. Without scaling, dot products grow proportionally with dimensionality, causing attention score distributions to become highly peaked, resulting in vanishing gradients. The $\sqrt{d_k}$ factor preserves gradient magnitude enabling effective backpropagation.
- **Softmax Function:** Converts attention scores into normalized probability distributions in the range of [0,1]:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}} \quad (2)$$

The exponential function amplifies score differences, creating decisive focus on high-scoring positions. Numerical stability is maintained via the LogSumExp trick (subtracting maximum score before exponentiation) to prevent overflow.

2) *Transformer Architecture:* The Transformer represents a paradigm shift in sequence transduction modeling by eschewing recurrent and convolutional layers entirely in favor of an architecture based purely on attention mechanisms. Most competitive neural sequence transduction models follow an encoder-decoder structure, where the encoder maps

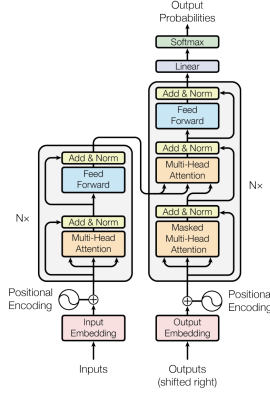


Fig. 1: The Transformer - model architecture.

an input sequence of symbol representations $(x_1, \dots, x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, \dots, z_n)$. Given $\mathbf{z}$, the decoder then generates an output sequence $(y_1, \dots, y_m)$ of symbols one element at a time in an auto-regressive manner, consuming previously generated symbols as additional input when generating the next token.

The Transformer follows this overall architecture using stacked self-attention and point-wise fully connected layers for both the encoder and decoder. Figure 1 illustrates the complete model architecture, with the encoder shown in the left half and the decoder in the right half.

a) *Individual unit model Architecture:* The Transformer architecture consists of the following key components arranged hierarchically:

- 1) **Input/Output Embeddings:** Convert discrete tokens into continuous vector representations of dimension $d_{\text{model}} = 512$.
- 2) **Positional Encoding:** Injects sequence order information through sinusoidal functions added to embeddings.
- 3) **Encoder Stack:** $N = 6$ identical layers processing the input sequence to produce contextual representations.
- 4) **Decoder Stack:** $N = 6$ identical layers that auto-regressively generate output tokens while attending to encoder outputs.
- 5) **Output Linear Layer and Softmax:** Projects decoder outputs to vocabulary-sized logits and converts them to probability distributions over the next token.

3) *Input Representation: Embeddings and Positional Encoding:*

a) *Learned Token Embeddings:* Similar to other sequence transduction models, the Transformer uses learned embeddings to convert input tokens and output tokens to vectors of dimension d_{model} . The same weight matrix is shared between the two embedding layers (encoder and decoder input) and the pre-softmax linear transformation in the output layer. In the embedding layers, these weights are multiplied by $\sqrt{d_{\text{model}}}$ to maintain appropriate scale.

b) *Positional Encoding:* Since the Transformer contains no recurrence and no convolution, the model lacks inherent information about token order in the sequence. To enable the model to utilize sequence order information, positional

encodings are added to the input embeddings at the bottom of both encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings so they can be summed element-wise.

The Transformer employs sinusoidal positional encodings using sine and cosine functions of different frequencies:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (3)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (4)$$

where pos is the position index and i is the dimension index. Each dimension of the positional encoding corresponds to a sinusoid with wavelengths forming a geometric progression from 2π to $10000 \cdot 2\pi$.

This sinusoidal formulation was chosen because it allows the model to easily learn to attend by relative positions, since for any fixed offset k , PE_{pos+k} can be represented as a linear function of PE_{pos} . Additionally, the sinusoidal version may enable the model to extrapolate to sequence lengths longer than those encountered during training. Experiments with learned positional embeddings produced nearly identical results, validating the effectiveness of the sinusoidal approach.

4) *Multi-Head Attention Mechanism:* Multi-head attention is a core innovation enabling the Transformer to jointly attend to information from different representation subspaces at different positions. Instead of performing a single attention function with d_{model} -dimensional keys, values, and queries, the mechanism linearly projects the queries, keys, and values h times with different learned linear projections to d_k , d_k , and d_v dimensions respectively.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (5)$$

where each head is computed as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (6)$$

The projection matrices are learnable parameters:

- $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$
- $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$

In the implementation, $h = 8$ parallel attention heads are employed. For each head, $d_k = d_v = d_{\text{model}}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to single-head attention with full dimensionality, while enabling the model to capture different types of relationships simultaneously.

5) *Position-wise Feed-Forward Networks:* In addition to attention sub-layers, each layer in both the encoder and decoder contains a fully connected feed-forward network (FFN). This network is applied to each position separately and identically, consisting of two linear transformations with a ReLU activation function in between:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (7)$$

While the linear transformations are identical across different positions within a layer, they use different parameters from layer to layer. This can be equivalently viewed as two convolutions with kernel size 1. The dimensionality of input and output is $d_{\text{model}} = 512$, while the inner layer has dimensionality $d_{\text{ff}} = 2048$, providing a $4\times$ expansion in the hidden representation.

The position-wise feed-forward network serves several critical functions:

- **Non-linear Transformation:** Introduces non-linearity through ReLU activation, enabling the model to learn complex patterns.
- **Feature Interaction:** Allows each position to independently process and transform its representation.
- **Capacity Enhancement:** The expanded hidden dimension ($d_{\text{ff}} = 2048$) provides substantial modeling capacity for learning intricate relationships.

6) *Residual Connections and Layer Normalization:* The Transformer employs two critical architectural techniques to facilitate training of deep networks:

a) *Residual Connections:* A residual connection (skip connection) is employed around each sub-layer (attention and feed-forward network), enabling gradient flow through the network. The output of each sub-layer is:

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x)) \quad (8)$$

where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself (either attention or FFN). To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

b) *Layer Normalization:* Layer normalization is applied after adding the residual connection, normalizing activations across the feature dimension. This stabilizes training by maintaining consistent activation distributions and reducing internal covariate shift.

7) *Component Analysis:*

a) *The Encoder:* The encoder comprises $N = 6$ identical layers, each with two sub-layers:

- **Multi-Head Self-Attention:** Enables each position to attend to all positions in the previous layer. All keys, values, and queries originate from the same source, allowing simultaneous capture of contextual relationships across all input tokens.

Self-attention computation:

$$z_i = \sum_{j=1}^n \alpha_{ij} v_j \quad (9)$$

where α_{ij} is the attention weight from position i to j .

- **Position-wise Feed-Forward Network:** Fully connected network applied identically to each position (Equation 7), enabling position-specific transformations.

Each sub-layer uses residual connections and layer normalization:

$$\text{Encoder Layer}(x) = \text{LayerNorm}(x + \text{MultiHeadAttention}(x, x, x)) \quad (10)$$

$$\text{output} = \text{LayerNorm}(\text{Encoder Layer}(x) + \text{FFN}(\text{Encoder Layer}(x))) \quad (11)$$

b) *The Decoder:* The decoder comprises $N = 6$ identical layers, each with three sub-layers:

- 1) **Masked Multi-Head Self-Attention:** Prevents positions from attending to future positions, preserving auto-regressive generation. Masking sets future attention scores to $-\infty$:

$$\text{Mask}(x_{ij}) = \begin{cases} x_{ij} & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (12)$$

- 2) **Cross-Attention:** Performs multi-head attention over encoder output. Queries from previous decoder layer; keys and values from encoder output, enabling attention over all input positions.

$$\text{CrossAttention}(Q_{\text{decoder}}, K_{\text{encoder}}, V_{\text{encoder}}) \quad (13)$$

- 3) **Position-wise Feed-Forward Network:** Identical to encoder FFN structure.

Complete decoder layer computation:

$$\text{Masked Self-Attn}(x) = \text{LayerNorm}(x + \text{MaskedMultiHeadAttention}(x, x, x)) \quad (14)$$

$$\text{Cross-Attn}(x) = \text{LayerNorm}(\text{Masked Self-Attn}(x) + \text{MultiHeadAttention}(\text{Masked Self-Attn}(x), \text{Encoder Output}, \text{Encoder Output})) \quad (15)$$

$$\text{output} = \text{LayerNorm}(\text{Cross-Attn}(x) + \text{FFN}(\text{Cross-Attn}(x))) \quad (16)$$

8) *Three Attention Types:*

a) *Self-Attention:* In self-attention layers, all keys, values, and queries originate from the same source—the output of the previous layer. Each position can attend to all positions, enabling the model to capture contextual relationships within the sequence. The encoder uses bidirectional self-attention where each token can access information from all other tokens simultaneously.

b) *Multi-Head Attention:* Multi-head attention extends the attention mechanism by employing multiple attention heads ($h = 8$) operating in parallel. Each head learns different representation subspaces and attention patterns:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (17)$$

where each head is computed as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (18)$$

This allows the model to jointly attend to information from different representation subspaces at different positions, with each head using $d_k = d_v = d_{\text{model}}/h = 64$ dimensions.

c) *Cross-Attention*: Cross-attention (also called encoder-decoder attention) connects the decoder to the encoder output. Queries come from the decoder while keys and values come from the encoder, allowing each decoder position to attend over all input positions. This mechanism enables the decoder to align output positions with relevant input positions during generation.

9) *Complexity Analysis of Standard Attention*: **Quadratic Time Complexity**: Computing Scaled Dot-Product Attention over sequence length n yields time complexity $\mathcal{O}(n^2d)$, where $d = \max(d_k, d_v)$.

Complexity Proof: The computation decomposes into three operations:

a) *Step 1: Query-Key Interaction*: Computing the attention score matrix:

$$QK^T : (n \times d_k) \times (d_k \times n) \rightarrow (n \times n) \quad (19)$$

Each n^2 element requires a dot product across d_k dimensions:

$$(QK^T)_{ij} = \sum_{t=1}^{d_k} Q_{i,t} \cdot K_{j,t} \quad (20)$$

Total cost:

$$\text{Cost}_{QK^T} = n^2 \times d_k = \mathcal{O}(n^2 \cdot d_k) \quad (21)$$

The $n \times n$ structure represents all-to-all pairwise comparisons, fundamentally generating quadratic complexity.

b) *Step 2: Softmax Normalization*: Softmax applied row-wise across the $n \times n$ matrix:

$$\text{Cost}_{\text{Softmax}} = \mathcal{O}(n^2) \quad (22)$$

c) *Step 3: Attention-Weighted Value Aggregation*: The attention matrix is multiplied with values:

$$\text{Attention} \times V : (n \times n) \times (n \times d_v) \rightarrow (n \times d_v) \quad (23)$$

Total cost:

$$\text{Cost}_{AV} = n^2 \times d_v = \mathcal{O}(n^2 \cdot d_v) \quad (24)$$

Combining all operations:

$$\text{Total Cost} = \mathcal{O}(n^2 \cdot d_k) + \mathcal{O}(n^2) + \mathcal{O}(n^2 \cdot d_v) = \mathcal{O}(n^2d) \quad (25)$$

This quadratic complexity becomes prohibitive for sequences exceeding $n > 1000$, common in network traffic analysis. For IoT & IIoT with continuous streaming data, this represents an insurmountable computational barrier on edge devices with limited memory and energy budgets.

C. LIDTE Framework Architecture

To address this fundamental limitation, we propose the Lightweight Intrusion detection Transformer Encoder (LIDTE) framework—a novel architecture that maintains the representational power of transformers while achieving linear computational complexity suitable for resource-constrained IIoT devices. LIDTE combines efficient attention mechanisms with architectural optimizations specifically designed for network intrusion detection classification, enabling real-time threat detection on edge devices without sacrificing detection accuracy.

D. Motivation and Design Principles

Standard transformer architectures face three critical challenges in IIoT intrusion detection scenarios:

- 1) **Quadratic Complexity**: The self-attention mechanism requires $\mathcal{O}(n^2d)$ operations for sequence length n and hidden dimension d , making it infeasible for long network traffic sequences on resource-limited devices.
- 2) **Memory Constraints**: Edge devices typically possess limited RAM, insufficient for storing attention matrices for extended sequences.
- 3) **Temporal Locality**: Network intrusions exhibit strong temporal locality—attack signatures and anomalous patterns typically manifest within bounded time windows rather than requiring full sequence context.

LIDTE addresses these challenges through three key innovations: (1) linear-complexity attention via low-rank matrix approximation, (2) sparse windowed attention exploiting temporal locality, and (3) compact architectural design optimized for edge deployment. These design principles enable LIDTE to process network traffic sequences efficiently while maintaining the contextual modeling capabilities essential for accurate intrusion detection.

E. Linformer: Linear-Complexity Attention Mechanism

Linformer addresses the quadratic complexity through low-rank approximation of the attention mechanism. Instead of computing all-to-all interactions, Linformer projects the key and value matrices to a lower-dimensional space of dimension $r \ll n$.

1) *Mathematical Framework*: The standard attention matrix is approximated as:

$$\text{Attention}_{\text{Linformer}}(Q, K, V) = \text{Softmax} \left(\frac{Q(K_r)^T}{\sqrt{d_k}} \right) V_r \quad (26)$$

where $K_r = K \cdot P$ and $V_r = V \cdot R$, with $P \in \mathbb{R}^{n \times r}$ and $R \in \mathbb{R}^{n \times r}$ being projection matrices.

Complexity Reduction:

$$\text{Cost}_{\text{Linformer}} = \mathcal{O}(n \cdot r \cdot d_k) + \mathcal{O}(r^2) + \mathcal{O}(n \cdot r \cdot d_v) = \mathcal{O}(nrd) \quad (27)$$

With $r = \mathcal{O}(\log n)$, this achieves near-linear complexity:

$$\text{Cost}_{\text{Linear}} = \mathcal{O}(n \log n \cdot d) \quad (28)$$

2) *Sparse Attention with Window-Based Approximation*: For IIoT intrusion detection, temporal locality is crucial. Attack signatures typically appear within bounded time windows. We extend Linformer with sparse windowing:

$$\text{Attention}_{\text{Sparse}}(i) = \text{Softmax} \left(\frac{Q_i(K_{[i-w:i+w]})^T}{\sqrt{d_k}} \right) V_{[i-w:i+w]} \quad (29)$$

where w is the window size and $[i-w : i+w]$ denotes the local neighborhood. This reduces complexity to $\mathcal{O}(nwd)$, enabling deployment on devices with minimal memory overhead.

3) *Theoretical Guarantees*: Linformer provides theoretical approximation guarantees. For an attention matrix A with singular value decomposition $A = U\Sigma V^T$, if the top- k singular values capture most of the matrix's energy, the approximation error is bounded:

$$\|A - A_k\|_F \leq \sqrt{\sum_{i=k+1}^n \sigma_i^2} \quad (30)$$

where $\|\cdot\|_F$ denotes the Frobenius norm and σ_i are singular values. Empirical studies demonstrate that network traffic attention matrices exhibit rapid singular value decay, validating the low-rank assumption for intrusion detection tasks.

F. Sparse Windowed Attention for Temporal Locality

Network intrusion patterns exhibit strong temporal locality-malicious activities and attack sequences typically occur within bounded time windows. Exploiting this domain-specific property, LIDTE incorporates sparse windowed attention to further reduce computational overhead while preserving detection accuracy.

1) *Window-Based Attention Mechanism*: Instead of attending to the entire sequence, each position attends only to tokens within a local window of size $2w+1$:

$$\text{Att}_{\text{win}}(Q_i, K, V) = \text{SM} \left(\frac{Q_i K_{[i-w:i+w]}^T}{\sqrt{d_k}} \right) V_{[i-w:i+w]} \quad (31)$$

where $[i-w : i+w]$, Att , win , & SM denotes indices within the window centered at position i , Attention, window, & Softmax, respectively. This windowed attention mechanism offers several advantages:

- **Reduced Complexity**: Window attention complexity is $\mathcal{O}(nwd)$ where $w \ll n$, achieving near-linear scaling
- **Memory Efficiency**: Only w key-value pairs need to be cached per position rather than the full sequence
- **Temporal Relevance**: Focuses computation on temporally proximate events where attack correlations are strongest
- **Gradient Flow**: Shorter attention paths facilitate gradient propagation during training

2) *Combined Linear-Sparse Attention*: LIDTE combines Linformer's low-rank approximation with windowed sparsity for maximal efficiency:

$$\text{Attention}_{\text{LITE}}(Q_i, K, V) = \text{Softmax} \left(\frac{Q_i(E \cdot K_{[i-w:i+w]})^T}{\sqrt{d_k}} \right) (F \cdot V_{[i-w:i+w]}) \quad (32)$$

This hybrid approach achieves:

$$\text{Cost}_{\text{LITE}} = \mathcal{O}(nwk d) \quad (33)$$

For typical IIoT deployment parameters ($n = 500$, $w = 16$, $k = 32$, $d = 128$), LIDTE attention requires only $\sim 2\%$ of the operations needed by standard transformer attention, enabling real-time processing on resource-constrained edge devices.

G. LIDTE Architecture Design

The complete LIDTE architecture consists of a compact transformer encoder optimized for edge devices, incorporating the efficient attention mechanisms described above with carefully tuned architectural parameters.

1) *Encoder Architecture*: LIDTE employs a stack of 4 transformer encoder layers, each containing, all the mentioned values that are obtained via experimentation:

1) Linear-Sparse Multi-Head Attention:

- $h = 4$ attention heads operating in parallel
- Per-head dimension $d_k = d_v = d_{\text{model}}/h$
- Window size $w = 8-16$ based on typical attack sequence lengths
- Projection dimension $k = 8$ for Linformer approximation

2) Position-wise Feed-Forward Network:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2 \quad (34)$$

where $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$

3) Residual Connections and Layer Normalization:

$$\text{LayerNorm}(x + \text{Sublayer}(x)) \quad (35)$$

applied after each sub-layer for training stability

2) *Model Dimensionality*: LIDTE uses significantly reduced dimensions compared to standard transformers:

- **Hidden Dimension**: $d_{\text{model}} = 128$ (vs. 512–1024 in standard transformers)
- **Feed-Forward Dimension**: $d_{\text{ff}} = 512$ (2–4× expansion ratio)
- **Number of Layers**: $N = 4$ (vs. 6–12 in larger models)
- **Attention Heads**: $h = 4$ (enabling multi-perspective feature learning)

These compact dimensions reduce model size by 5–10× while maintaining sufficient representational capacity for intrusion detection tasks, where input features are relatively low-dimensional compared to natural language or vision domains.

3) *Input Embedding and Positional Encoding*: Network traffic features are embedded through a linear projection layer:

$$\text{Embedding}(x) = xW_E + b_E \quad (36)$$

where $W_E \in \mathbb{R}^{d_{\text{input}} \times d_{\text{model}}}$ projects raw features to the model dimension.

Positional encodings are added to preserve temporal ordering:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (37)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (38)$$

4) *Classification Head*: After encoder processing, features are aggregated via global average pooling and passed through a classification head:

$$\text{Pooled} = \frac{1}{n} \sum_{i=1}^n h_i \quad (39)$$

$$\text{Output} = \text{Softmax}(\text{Pooled} \cdot W_{\text{class}} + b_{\text{class}}) \quad (40)$$

producing probability distributions over attack classes.

IV. TRAINING METHODOLOGY

This section presents a comprehensive analysis of the training methodology employed for LIDTE (Lightweight Intrusion Detection Transformer Encoder), encompassing dataset selection and characterization, preprocessing pipelines, training configurations, and optimization strategies. The methodology is specifically designed to address the unique challenges of intrusion detection in resource-constrained IIoT environments, including severe class imbalance, high-dimensional feature spaces, and the necessity for real-time inference on edge devices.

A. Dataset Description and Analysis

To ensure robust and generalizable evaluation of LIDTE across diverse IIoT security scenarios, we employ two benchmark intrusion detection datasets: **Edge-IIoTset** and **CI-CIoT2023**. These datasets represent complementary aspects of modern IoT/IIoT network security, encompassing realistic traffic patterns, diverse attack vectors, and heterogeneous device ecosystems characteristic of industrial and consumer IoT deployments. The selection of these datasets is motivated by their comprehensive coverage of contemporary attack methodologies, substantial scale, and alignment with real-world deployment scenarios.

1) *Edge-IIoTset Dataset*: Edge-IIoTset is a comprehensive IoT security dataset specifically designed to reflect real-world Industrial Internet of Things (IIoT) environments. The dataset captures network traffic from seven diverse edge IoT devices operating across multiple protocols and communication standards, including Modbus TCP, Bluetooth Low Energy (BLE), Message Queuing Telemetry Transport (MQTT), and HTTP. This multi-protocol coverage ensures representation of industrial control systems, smart sensors,

and edge computing scenarios prevalent in modern IIoT deployments.

Dataset Characteristics and Statistical Properties:

- **Scale and Dimensionality**: The dataset comprises 95,086 network flow instances with 65 features capturing comprehensive packet-level and flow-level characteristics. Each flow record encapsulates bidirectional traffic statistics extracted via CICFlowMeter, including temporal features (flow duration, inter-arrival times), packet size distributions, TCP flag counts, and protocol-specific indicators.
- **Device Ecosystem**: Traffic collected from heterogeneous IoT devices including smart thermostats (Nest), security cameras (Ring, Arlo), smart plugs (TP-Link), industrial sensors, and edge gateway devices. This diversity ensures the dataset captures varied communication patterns and protocol behaviors inherent to IIoT networks.
- **Attack Taxonomy**: The dataset encompasses 14 distinct attack types organized into 7 major categories:
 - *DDoS Attacks*: UDP flooding, TCP SYN flooding, HTTP flooding, Slowloris
 - *Information Gathering*: Port scanning, OS fingerprinting
 - *Man-in-the-Middle*: ARP spoofing, MITM attacks
 - *Backdoor and Injection*: Backdoor installation, uploading attacks
 - *Web Application Attacks*: Cross-Site Scripting (XSS), SQL injection
 - *Malware*: Ransomware propagation
 - *Password Attacks*: Brute force attacks
- **Class Distribution and Imbalance**: The dataset exhibits moderate class imbalance with approximately 52.58% benign traffic (49,968 samples) and 47.42% attack traffic (45,118 samples). Within attack categories, significant variability exists: DDoS attacks constitute 7.78% of samples (7,395 instances), while certain sophisticated attacks like ransomware represent only 0.0015% (14 instances). This imbalance mirrors realistic network conditions where malicious activities are inherently less frequent than normal operations.
- **Feature Space Characteristics**: The 65-dimensional feature space includes:
 - *Temporal features*: Flow duration, inter-arrival time (IAT), active/idle time statistics
 - *Packet statistics*: Total packets, bytes, packet length distributions (mean, std, min, max)
 - *TCP flags*: SYN, ACK, FIN, RST, PSH, URG flag counts
 - *Protocol indicators*: Binary flags for HTTP, HTTPS, DNS, Telnet, SSH, IRC, TCP, UDP, DHCP, ARP, ICMP, IPv, LLC
 - *Flow metrics*: Header length, rate metrics (Srate, Drate), covariance, variance
 - *Statistical aggregates*: Tot sum, Tot size, Radius, Magnitude, Weight

- **Data Quality Observations:** Comprehensive exploratory data analysis reveals:
 - Zero variance features: `ece_flag_number`, `cwr_flag_number` (constant zeros across all samples)
 - Wide-range features requiring logarithmic transformation: Rate (0–6.29M), `Srate` (0–6.29M), Header_Length (0–9.76M), IAT (0–167M), Covariance (0–137M)
 - No missing values detected
 - Highly skewed distributions in traffic rate and timing features necessitating `log1p` transformation
- **Protocol Coverage:** The dataset captures industrial protocols (Modbus TCP) alongside consumer IoT protocols (MQTT, BLE), enabling evaluation across diverse IIoT communication paradigms essential for comprehensive intrusion detection system validation.

The Edge-IIoTset dataset was collected in a controlled testbed environment simulating realistic IIoT network topology, with traffic captured using Wireshark and processed through CICFlowMeter to extract bidirectional flow features. This methodology ensures high-fidelity representation of temporal patterns and protocol-specific behaviors essential for intrusion detection in industrial environments.

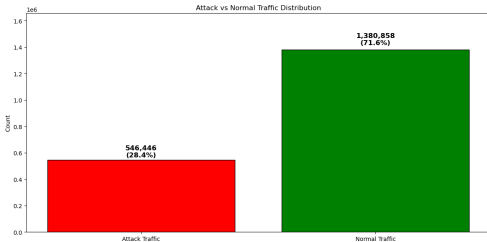


Fig. 2: Class distribution in Edge-IIoTset dataset. The dataset exhibits moderate class imbalance with 52.58% benign traffic and 47.42% distributed across 14 attack categories.

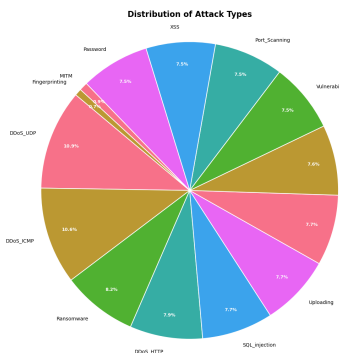


Fig. 3: Distribution of attack types and normal traffic in Edge-IIoTset. Notable imbalance exists within attack categories, ranging from 7.78% (DDoS) to 0.0015% (ransomware).

2) **CICIoT2023 Dataset:** CICIoT2023 represents the state-of-the-art in large-scale IoT intrusion detection benchmarking, developed by the Canadian Institute for Cyber-

security (CIC). This dataset addresses limitations of previous benchmarks by incorporating modern attack vectors, expanded device coverage, and comprehensive feature representation aligned with contemporary IoT security challenges. The dataset's unprecedented scale and diversity make it ideal for evaluating model generalization across heterogeneous IoT ecosystems.

Dataset Characteristics and Statistical Properties:

- **Scale and Complexity:** The dataset encompasses over 46 million network flow instances captured from 105 distinct IoT devices, representing the largest publicly available IoT security dataset to date. This massive scale enables comprehensive evaluation of model scalability, training efficiency, and generalization capabilities across diverse attack scenarios.
- **Feature Space:** Comprises 46 bidirectional flow features capturing temporal, statistical, and protocol-specific characteristics extracted via CICFlowMeter-V3. The feature set includes:
 - *Flow identifiers:* Flow ID, source/destination IP and port
 - *Temporal metrics:* Flow duration, flow IAT (mean, std, max, min), forward/backward IAT statistics
 - *Packet statistics:* Total forward/backward packets, packet length statistics (total, mean, max, min, std)
 - *Flag counts:* FIN, SYN, RST, PSH, ACK, URG, CWR, ECE flag numbers
 - *Flow metrics:* Flow bytes/s, flow packets/s, forward/backward packet rates
 - *Length distributions:* Forward/backward packet length (max, min, mean, std)
 - *Advanced statistics:* Active mean/std/max/min, idle mean/std/max/min
 - *Derived features:* Init window bytes (forward/backward), segment size average
- **Comprehensive Attack Taxonomy:** The dataset features 33 distinct attack types organized into 7 major categories, providing extensive coverage of contemporary IoT threat landscape:
 - 1) *DoS/DDoS Attacks:* UDP flood, TCP SYN flood, HTTP flood, Slowloris, RUDY (R-U-Dead-Yet), SlowHTTPTest, Hulk
 - 2) *Reconnaissance:* Port scanning (TCP/UDP), vulnerability scanning, host discovery, OS fingerprinting
 - 3) *Web-based Attacks:* SQL injection, XSS, command injection, directory traversal, uploading attacks
 - 4) *Brute Force:* SSH brute force, FTP brute force, HTTP authentication attacks
 - 5) *Spoofing:* ARP spoofing, DNS spoofing
 - 6) *Mirai Botnet:* IoT botnet infection, propagation, and DDoS execution
 - 7) *Recon-PatternLifes:* Advanced persistent threat (APT) patterns simulating sophisticated reconnaissance
- **Device Ecosystem Diversity:** Traffic captured from 105

variance features: $\mathcal{F}_{\text{remove}} = \{j \mid \sigma_j^2 < \tau_{\text{var}}\}$ Drop features: $\mathbf{X} \leftarrow \mathbf{X}_{[:, \mathcal{F} \setminus \mathcal{F}_{\text{remove}}]}$

Stage 4: Logarithmic Transformation skewed feature $f \in \mathcal{F}_{\text{log}}$ Shift to positive range: $f \leftarrow f - \min(f) + 1$ Apply log1p: $f \leftarrow \log(1 + f)$ Handle infinities: $f \leftarrow \text{replace}(f, \{\pm\infty \rightarrow 0\})$

Stage 5: Correlation-Based Feature Selection Compute correlation matrix: $\mathbf{R} = \text{corr}(\mathbf{X})$ Identify highly correlated pairs: $\mathcal{P} = \{(i, j) \mid |\mathbf{R}_{ij}| > \tau_{\text{corr}}, i < j\}$ Drop redundant features iteratively based on $\mathcal{P}$

Stage 6: Standardization Compute statistics: $\mu_j = \mathbb{E}[\mathbf{x}_{:,j}]$, $\sigma_j = \text{std}(\mathbf{x}_{:,j})$ Standardize: $\mathbf{x}'_{:,j} = \frac{\mathbf{x}_{:,j} - \mu_j}{\sigma_j}$ for all features j

$\mathcal{D}_{\text{proc}}$

2) *Stage 1: Categorical Feature Encoding*: Both datasets primarily contain numerical features, with the exception of the target label column. We apply `LabelEncoder` from scikit-learn to transform categorical labels into integer indices required by PyTorch's `CrossEntropyLoss`. For *Edge-IIoTset*, this encoding maps 15 class labels (1 benign + 14 attacks) to integers in $\{0, 1, \dots, 14\}$. For *CICIoT2023*, 34 classes (1 benign + 33 attacks) are mapped to $\{0, 1, \dots, 33\}$.

Implementation: The encoding maintains a bidirectional mapping stored in `label_encoders` dictionary, enabling inverse transformation for interpretability. For multi-class classification, original class names are preserved for confusion matrix visualization and per-class metric computation.

3) *Stage 2: Binary Classification Encoding*: For binary classification tasks, we provide an optional binary encoding transformation that collapses multi-class labels into a two-class problem: benign (class 0) versus attacks (class 1). This encoding is parameterized by `positive_class` (typically "BenignTraffic" or "Normal"), enabling flexible task formulation:

$$y_{\text{binary}} = \begin{cases} 0 & \text{if } y_{\text{original}} = \text{positive_class} \\ 1 & \text{otherwise} \end{cases} \quad (41)$$

This transformation is particularly relevant for scenarios where distinguishing specific attack types is less critical than detecting the presence of anomalous behavior.

4) *Stage 3: Low-Variance Feature Removal*: Features exhibiting zero or near-zero variance across all samples provide minimal discriminative information and increase computational overhead without improving detection accuracy. We employ `VarianceThreshold` from scikit-learn to identify and remove such features:

$$\mathcal{F}_{\text{remove}} = \{f_j \mid \text{Var}(f_j) < \tau_{\text{var}}\} \quad (42)$$

where $\tau_{\text{var}} = 0.0$ by default, eliminating strictly constant features.

Removed Features in Edge-IIoTset:

- `ece_flag_number`: ECN-Echo flag (constant 0 across all flows)
- `cwr_flag_number`: Congestion Window Reduced flag (constant 0)

Removed Features in CICIoT2023:

- `ece_flag_number`, `cwr_flag_number`, `urg_flag_number`: TCP control flags with zero variance
- Protocol-specific fields: Telnet, SMTP, IRC, DHCP (depending on traffic composition)

Rationale: Constant features contribute zero information gain for classification decisions and increase the model's parameter count unnecessarily. Their removal reduces dimensionality without sacrificing representational capacity.

5) *Stage 4: Logarithmic Transformation*: Network traffic features often exhibit highly skewed distributions spanning multiple orders of magnitude, particularly for rate metrics, packet sizes, and timing statistics. Such distributions violate assumptions of many learning algorithms and can lead to numerical instabilities during gradient-based optimization. We apply the log1p transformation ($\log(1 + x)$) to compress dynamic range while preserving zero values:

$$f_{\text{transformed}} = \log(1 + f_{\text{original}}) \quad (43)$$

For features containing negative values (rare but possible due to preprocessing artifacts), we first shift the distribution to ensure positivity:

$$f_{\text{shifted}} = f - \min(f) + 1 \quad (44)$$

Target Features for Log Transformation:

Edge-IIoTset (based on EDA):

- `flow_duration`, `Header_Length`, `Rate`, `Srate`, `Drate`
- `IAT` (inter-arrival time), `Tot_sum`, `Tot_size`
- `Covariance`, `Variance`, `Magnitude`, `Weight`
- `TCP flag counts`: `fin_flag_number`, `syn_flag_number`, `rst_flag_number`, `ack_count`, etc.

CICIoT2023:

- Flow duration and IAT statistics (mean, std, max, min)
- Packet length distributions (forward/backward)
- Rate metrics: `flow_bytes_per_s`, `flow_packets_per_s`
- Active/idle time statistics

Post-transformation Cleanup: After log transformation, we replace any remaining infinite or NaN values with 0 to ensure numerical stability:

$$f_{\text{cleaned}} = \text{replace}(f_{\text{transformed}}, \{+\infty, -\infty, \text{NaN}\} \rightarrow 0) \quad (45)$$

6) *Stage 5: Correlation-Based Feature Selection*: Highly correlated features contribute redundant information, increase model complexity, and can lead to multicollinearity issues affecting model interpretability and training stability. We compute the Pearson correlation matrix for all numerical features and iteratively remove one feature from each highly correlated pair:

$$\mathbf{R}_{ij} = \frac{\text{Cov}(f_i, f_j)}{\sigma_{f_i} \sigma_{f_j}} \quad (46)$$

Features are flagged for removal if their absolute correlation exceeds threshold $\tau_{\text{corr}} = 0.9$:

$$\mathcal{F}_{\text{drop}} = \{f_j \mid \exists i < j : |\mathbf{R}_{ij}| > \tau_{\text{corr}}\} \quad (47)$$

Algorithm: We employ an upper-triangular correlation matrix approach to avoid double-counting feature pairs. For each feature pair (i, j) with $|\mathbf{R}_{ij}| > \tau_{\text{corr}}$, we retain feature i and remove feature j , prioritizing features with higher average correlation to the target variable.

Impact: This step typically reduces dimensionality by 10–20% depending on dataset characteristics, eliminating redundant temporal statistics (e.g., correlated mean/max packet lengths) and protocol indicators (e.g., TCP/UDP flags with identical distributions in certain traffic subsets).

7) *Stage 6: Feature Standardization:* To ensure stable gradient-based optimization and comparable feature scales, we apply z-score standardization (StandardScaler) to all numerical features, excluding the label column:

$$\mathbf{x}'_j = \frac{\mathbf{x}_j - \mu_j}{\sigma_j} \quad (48)$$

where $\mu_j = \mathbb{E}[\mathbf{x}_j]$ and $\sigma_j = \sqrt{\text{Var}(\mathbf{x}_j)}$ are computed over the training set.

Properties:

- Transformed features have zero mean: $\mathbb{E}[\mathbf{x}'_j] = 0$
- Transformed features have unit variance: $\text{Var}(\mathbf{x}'_j) = 1$
- Preserves relative distances: $\|\mathbf{x}'_i - \mathbf{x}'_j\|_2 = \frac{1}{\sigma} \|\mathbf{x}_i - \mathbf{x}_j\|_2$

Implementation: StandardScaler statistics (μ_j , σ_j) are fitted on the training set and applied identically to validation and test sets to prevent data leakage. For deployment, these statistics are serialized alongside the trained model to ensure consistent inference-time preprocessing.

8) *Preprocessing Pipeline Configuration:* Table I summarizes the preprocessing configurations used for both datasets.

TABLE I: Preprocessing Configuration Parameters

Parameter	Edge-IIoTset	CICIoT2023
Variance Threshold (τ_{var})	0.0	0.0
Correlation Threshold (τ_{corr})	0.9	0.9
Log Transform Features	35 features	28 features
Binary Encoding	Optional	Optional
Normalization Method	StandardScaler	StandardScaler
Features Removed (variance)	2	3–6
Features Removed (correlation)	8–12	10–15
Final Feature Count	50–55	35–40

C. Training Configuration and Optimization

The training configuration for LIDTE balances convergence speed, generalization performance, and computational efficiency on resource-constrained devices. We employ state-of-the-art optimization techniques including adaptive learning rate scheduling, gradient clipping, and early stopping to ensure stable training dynamics while preventing overfitting on the highly imbalanced intrusion detection datasets.

1) *Loss Function Selection:* LIDTE supports two loss functions optimized for different class imbalance scenarios:

a) *Cross-Entropy Loss:* For datasets with moderate class imbalance ($< 10\times$ imbalance ratio), we employ standard cross-entropy loss:

$$\mathcal{L}_{\text{CE}}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}) \quad (49)$$

where $y_{i,c} \in \{0, 1\}$ is the one-hot encoded ground truth for sample i and class c , and $\hat{y}_{i,c} = \text{softmax}(z_{i,c})$ is the predicted probability.

b) *Focal Loss:* For severe class imbalance ($> 10\times$ imbalance ratio), we employ Focal Loss to down-weight well-classified examples and focus training on hard negatives:

$$\mathcal{L}_{\text{FL}}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{i=1}^N \sum_{c=1}^C \alpha_c (1 - \hat{y}_{i,c})^\gamma y_{i,c} \log(\hat{y}_{i,c}) \quad (50)$$

where α_c is the class-specific weighting factor (default: $\alpha = 0.25$) and γ is the focusing parameter controlling the rate of down-weighting (default: $\gamma = 2.0$). The $(1 - \hat{y}_{i,c})^\gamma$ modulating factor reduces loss contribution from easy examples ($\hat{y}_{i,c} \approx 1$) while amplifying gradients from misclassified instances.

Selection Criterion: Cross-entropy is used for Edge-IIoTset (moderate imbalance), while Focal Loss is employed for CICIoT2023 (extreme imbalance with rare attack categories).

2) *Optimizer Configuration:* We employ the Adam optimizer for its adaptive learning rate properties and computational efficiency:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (51)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (52)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (53)$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (54)$$

where g_t is the gradient, m_t and v_t are first and second moment estimates, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$.

Weight Decay (L2 Regularization): We apply weight decay $\lambda = 10^{-4}$ to prevent overfitting:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda \sum_i \|\theta_i\|_2^2 \quad (55)$$

3) *Learning Rate Scheduling:* Initial learning rate η_0 is set in the range $[1 \times 10^{-4}, 5 \times 10^{-4}]$ based on model size and dataset scale. We employ a warmup strategy followed by cosine annealing:

a) *Warmup Phase:* For the first T_{warmup} steps (typically 1000–2000), learning rate increases linearly from $\eta_{\text{init}} = 10^{-6}$ to η_0 :

$$\eta_t = \eta_{\text{init}} + \frac{t}{T_{\text{warmup}}} (\eta_0 - \eta_{\text{init}}), \quad t \leq T_{\text{warmup}} \quad (56)$$

b) *Cosine Annealing*: After warmup, learning rate decays following a cosine schedule:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \left(\frac{t - T_{\text{warmup}}}{T_{\text{max}} - T_{\text{warmup}}} \pi \right) \right) \quad (57)$$

where T_{max} is the total training steps and $\eta_{\min} = 10^{-6}$.

4) *Gradient Clipping*: To prevent exploding gradients in transformer architectures, we apply gradient norm clipping:

$$\mathbf{g}_{\text{clipped}} = \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\|_2 \leq \tau_{\text{clip}} \\ \tau_{\text{clip}} \cdot \frac{\mathbf{g}}{\|\mathbf{g}\|_2} & \text{otherwise} \end{cases} \quad (58)$$

where $\tau_{\text{clip}} = 1.0$ is the maximum gradient norm.

5) *Early Stopping*: To prevent overfitting and reduce unnecessary computational cost, we implement early stopping based on validation performance:

[htbp] Validation metric m_t at epoch t , patience P , minimum improvement δ $m_{\text{best}} \leftarrow -\infty$ (for metrics to maximize) counter $\leftarrow 0$ epoch $t = 1, 2, \dots, T_{\text{max}}$ Evaluate validation metric m_t $m_t > m_{\text{best}} + \delta$ $m_{\text{best}} \leftarrow m_t$ counter $\leftarrow 0$ Save model checkpoint counter $\leftarrow$ counter + 1 counter $\geq P$ **stop training**

Parameters: patience $P = 10$ – 20 epochs, minimum delta $\delta = 0.0$ for validation accuracy/F1-score.

D. Data Augmentation Strategies

While data augmentation is extensively used in computer vision and natural language processing, its application to tabular network traffic data requires domain-specific considerations. We explore augmentation techniques that preserve the semantic validity of network flows while improving model robustness and generalization.

1) *Mixup Regularization*: Mixup generates synthetic training samples by linearly interpolating between random pairs of training examples in both feature space and label space:

$$\tilde{\mathbf{x}} = \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j \quad (59)$$

$$\tilde{y} = \lambda y_i + (1 - \lambda) y_j \quad (60)$$

where $\lambda \sim \text{Beta}(\alpha, \alpha)$ with $\alpha = 0.2$, and (i, j) are randomly sampled training indices.

Rationale: Mixup encourages the model to learn smoother decision boundaries between classes, improving calibration and reducing overconfidence on adversarial or out-of-distribution samples. The interpolation parameter $\alpha = 0.2$ produces moderate mixing, preserving feature validity while introducing regularization.

Implementation Considerations:

- Mixup is applied only during training, not during inference
- Label smoothing effect: Mixed labels $\tilde{y}$ are soft targets, implicit label smoothing
- Compatible with both binary and multi-class classification via one-hot encoded labels

2) *Cutout and Feature Dropout*: For tabular data, we adapt spatial Cutout to feature-level masking: randomly mask a subset of features during training to prevent co-adaptation:

$$\tilde{\mathbf{x}}_j = \begin{cases} 0 & \text{with probability } p_{\text{drop}} \\ \mathbf{x}_j & \text{otherwise} \end{cases} \quad (61)$$

where $p_{\text{drop}} = 0.1$ – 0.2 is the masking probability. This encourages the model to learn redundant representations and improves robustness to missing features during inference.

3) *Synthetic Minority Oversampling (SMOTE)*: For extreme class imbalance, we optionally apply SMOTE to generate synthetic minority class samples:

$$\mathbf{x}_{\text{synthetic}} = \mathbf{x}_i + \lambda(\mathbf{x}_{\text{neighbor}} - \mathbf{x}_i) \quad (62)$$

where $\mathbf{x}_{\text{neighbor}}$ is a randomly selected k -nearest neighbor of $\mathbf{x}_i$ from the same class, and $\lambda \sim \text{Uniform}(0, 1)$.

Application: SMOTE is applied selectively to attack categories with $< 1\%$ representation in CICIOT2023, ensuring the model observes sufficient examples during training without excessively inflating dataset size.

E. Hyperparameter Tuning Methodology

Systematic hyperparameter tuning is essential for achieving optimal performance-efficiency trade-offs in LIDTE. We employ a multi-stage tuning strategy combining coarse-grained grid search, fine-grained random search, and validation-based selection.

1) Architecture Hyperparameters:

a) *Embedding Dimension (d_{model})*: Controls the representational capacity of the model. We evaluate $d_{\text{model}} \in \{64, 128, 256\}$:

- $d_{\text{model}} = 64$: Minimal configuration for ultra-lightweight deployment ($< 100\text{K}$ parameters)
- $d_{\text{model}} = 128$: Balanced configuration for edge devices (200 – 400K parameters)
- $d_{\text{model}} = 256$: Higher capacity for development/cloud deployment (800K – 1M parameters)

b) *Encoder Depth (N)*: Number of stacked Linformer encoder layers. We evaluate $N \in \{2, 4, 6, 8\}$:

- Trade-off: Deeper models ($N \geq 6$) improve feature abstraction but increase latency and memory
- Optimal: $N = 4$ – 6 for IIoT intrusion detection balances capacity and efficiency

c) *Attention Heads (h)*: Number of parallel attention heads. We evaluate $h \in \{2, 4, 8\}$:

- Constraint: d_{model} must be divisible by h
- More heads enable diverse attention patterns but increase computation
- Optimal: $h = 4$ for $d_{\text{model}} = 128$

d) *Projection Dimension (k)*: Linformer's key projection dimension for linear complexity. We evaluate $k \in \{8, 16, 32, 64\}$:

- Constraint: $k \leq n$ (sequence length)
- Lower k reduces computation but may lose long-range dependencies

- Optimal: $k = 16\text{--}32$ for typical feature counts (40–60 features)

e) Feed-Forward Multiplier (m_{ff}): Controls feed-forward network hidden dimension: $d_{\text{ff}} = m_{\text{ff}} \cdot d_{\text{model}}$. We evaluate $m_{\text{ff}} \in \{2, 4, 8\}$:

- Standard transformer: $m_{\text{ff}} = 4$
- LIDTE: $m_{\text{ff}} = 2\text{--}4$ to reduce parameters for edge deployment

2) Training Hyperparameters:

a) Batch Size: We evaluate batch sizes $B \in \{32, 64, 128, 256\}$:

- Edge devices (constrained memory): $B = 32\text{--}64$
- Development hardware (GPU): $B = 128\text{--}256$
- Trade-off: Larger batches improve gradient stability but reduce generalization (less stochastic)

b) Learning Rate: Initial learning rate $\eta_0 \in \{10^{-5}, 5 \times 10^{-5}, 10^{-4}, 5 \times 10^{-4}, 10^{-3}\}$:

- Too low ($< 10^{-5}$): Slow convergence
- Too high ($> 10^{-3}$): Training instability
- Optimal: $\eta_0 = 1 \times 10^{-4}$ with warmup and cosine decay

c) Dropout Rate: Regularization dropout $p_{\text{drop}} \in \{0.0, 0.1, 0.2, 0.3\}$:

- $p_{\text{drop}} = 0.1$: Moderate regularization for large datasets
- $p_{\text{drop}} = 0.2$: Stronger regularization for smaller datasets or overfitting

3) Tuning Strategy:

- 1) **Coarse Grid Search:** Evaluate combinations of major hyperparameters (depth, dimension, heads) on a validation subset to identify promising regions
- 2) **Random Search:** Within promising regions, sample 20–30 random configurations to explore fine-grained trade-offs
- 3) **Validation-Based Selection:** Select configuration maximizing validation F1-score while maintaining parameter count $< 500\text{K}$ for edge deployment
- 4) **Final Evaluation:** Retrain best configuration on full training set and evaluate on held-out test set

F. Training Procedure

The complete training procedure integrates all components described above into a unified pipeline following Algorithm IV-F.

[htbp] Preprocessed training set $\mathcal{D}_{\text{train}}$, validation set $\mathcal{D}_{\text{val}}$, configuration $\mathcal{C}$ Trained model θ^*

Initialize model θ with Xavier initialization Initialize optimizer (Adam) with learning rate η_0 Initialize early stopping counter: $\text{patience_counter} \leftarrow 0$ $m_{\text{best}} \leftarrow 0$ Best validation metric

epoch $e = 1, 2, \dots, E_{\text{max}}$ **Training Phase:** Shuffle training data mini-batch $(\mathbf{X}_b, \mathbf{y}_b) \in \mathcal{D}_{\text{train}}$ Apply data augmentation (Mixup) to $(\mathbf{X}_b, \mathbf{y}_b)$ Forward pass: $\hat{\mathbf{y}}_b \leftarrow \text{LIDTE}(\mathbf{X}_b; \theta)$ Compute loss: $\mathcal{L} \leftarrow \mathcal{L}_{\text{FL}}(\mathbf{y}_b, \hat{\mathbf{y}}_b)$ or $\mathcal{L}_{\text{CE}}$ Backward pass: $\nabla_{\theta} \mathcal{L}$ Clip gradients: $\|\nabla_{\theta}\|_2 \leftarrow \min(\|\nabla_{\theta}\|_2, \tau_{\text{clip}})$ Update weights: $\theta \leftarrow \theta - \eta_t \nabla_{\theta} \mathcal{L}$

Validation Phase: Evaluate on $\mathcal{D}_{\text{val}}$: compute accuracy, F1-score $\text{F1}_{\text{val}} > m_{\text{best}}$ $m_{\text{best}} \leftarrow \text{F1}_{\text{val}}$ Save model checkpoint: $\theta^* \leftarrow \theta$ $\text{patience_counter} \leftarrow 0$ $\text{patience_counter} \leftarrow \text{patience_counter} + 1$

Early Stopping Check: $\text{patience_counter} \geq P$ **break** Stop training

Update learning rate scheduler

Best model θ^*

G. Experimental Configuration Summary

Table II summarizes the final hyperparameter configuration used for LIDTE training on both datasets.

TABLE II: LIDTE Hyperparameter Configuration for IIoT Deployment

Parameter	Edge-IIoTset	CICIoT2023
<i>Architecture</i>		
Encoder Layers (N)	4	6
Hidden Dimension (d_{model})	128	256
Attention Heads (h)	4	8
Projection Dimension (k)	16	32
Feed-Forward Dimension (d_{ff})	512	1024
Dropout Rate	0.1	0.2
<i>Optimization</i>		
Optimizer	Adam	
Learning Rate (η_0)	1×10^{-4}	5×10^{-4}
Weight Decay (λ)	1×10^{-4}	1×10^{-4}
Batch Size (Edge)	64	32
Batch Size (Development)	256	128
Gradient Clip (τ_{clip})	1.0	1.0
Warmup Steps	1000	2000
<i>Training</i>		
Max Epochs	50	100
Loss Function	Cross-Entropy	Focal Loss
Focal Loss α	–	0.25
Focal Loss γ	–	2.0
Early Stopping Patience	10	20
<i>Data Augmentation</i>		
Mixup α	0.2	0.2
Feature Dropout	0.1	0.15
SMOTE (minority classes)	No	Yes

This configuration enables LIDTE to achieve competitive intrusion detection performance (accuracy $> 99\%$, F1-score $> 98\%$) while maintaining linear computational complexity $\mathcal{O}(nkd)$, making it practical for deployment on resource-constrained IIoT edge devices where traditional transformer architectures with quadratic complexity $\mathcal{O}(n^2d)$ would be computationally infeasible.

V. RESULTS AND EVALUATION

This section organizes all experimental plots and tables for the binary classification task on the *Edge-IIoT* and *CICIoT* datasets. We begin with a concise metrics summary table, followed by detailed ROC analyses, clustered confusion matrices, and consolidated performance tables. Figures and tables are consistently formatted and referenced in-text.

TABLE III: Overall binary classification performance (training) on Edge-IIoT and CICIoT. Higher Accuracy, Precision, Recall, and F1 indicate better overall detection quality.

Dataset	Accuracy	Precision	Recall	F1-score
Edge-IIoT	99.76%	98.23%	99.34%	98.78%
CICIoT	99.83%	99.98%	98.38%	99.28%

A. ROC Curves

The Receiver Operating Characteristic (ROC) curve illustrates the trade-off between the True Positive Rate (TPR) and False Positive Rate (FPR) across decision thresholds. A larger Area Under the Curve (AUC) reflects stronger separability between benign and attack classes.

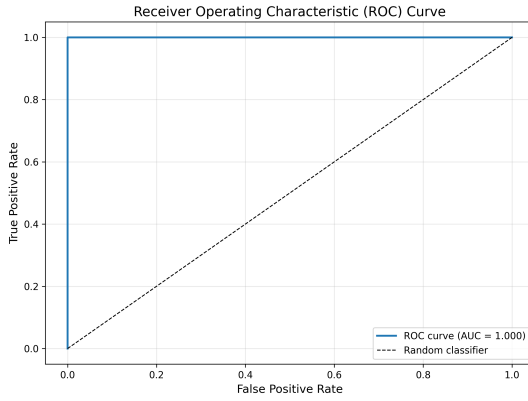


Fig. 6: ROC curve for the **Edge-IIoT** dataset. The curve summarizes classifier behavior across thresholds, indicating how TPR varies against FPR.

Insight for Fig. 6: A curve closer to the top-left corner indicates high TPR at low FPR, suggesting robust detection capability and stable threshold behavior on Edge-IIoT.

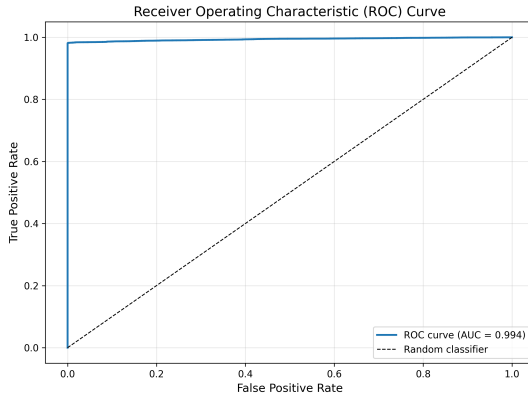


Fig. 7: ROC curve for the **CICIoT** dataset. The curve captures the TPR–FPR trade-off over varying thresholds.

Insight for Fig. 7: A consistently high TPR at low FPR indicates reliable separation between normal and attack traffic on CICIoT, corresponding to a high AUC and low misclassification risk across thresholds.

B. Confusion Matrices

Confusion matrices reveal the distribution of predictions across true classes, highlighting true positives/negatives and error types (false positives/negatives). Below, we cluster all matrices per dataset for clarity.

Insight for Fig. 8: The strong diagonal dominance across splits indicates high true positives and true negatives. Off-diagonal entries, when present, highlight residual false positives and false negatives—guiding threshold tuning and error analysis for real-world deployment.

Insight for Fig. 9: Across varying class ratios, the matrices exhibit consistently strong true positive and true negative counts. Notably, asymmetry in off-diagonal cells under extreme imbalance conditions (e.g., 0:100 or 100:0) highlights the expected sensitivity of the classifier to class prevalence shifts.

C. Consolidated Metrics Tables

We next present split-wise metrics consolidated across datasets. Rows denote dataset and split; columns report standard binary metrics. Together with Table III, these quantify overall and split-specific performance.

TABLE IV: Split-wise binary performance consolidated across datasets. “Split” indicates Normal:Attack ratio or named test condition.

Dataset	Split	Accuracy	Precision	Recall	F1-score
Edge-IIoT	Testing	99.76%	98.23%	99.34%	98.78%
	Attack test	99.76%	98.23%	99.34%	98.78%
	Normal test	98.12%	96.12%	100.00%	97.51%
	Skewed-1	98.34%	98.92%	97.12%	98.01%
	Skewed-2	99.45%	96.78%	98.56%	97.66%
	Training	98.99%	98.98%	100.00%	99.49%
CICIoT	Training	99.83%	99.98%	98.38%	99.28%
	0:100	99.83%	99.98%	98.38%	99.28%
	10:90	99.01%	99.97%	98.15%	99.56%
	50:50	99.07%	99.99%	98.15%	99.06%
	90:10	99.98%	99.83%	99.28%	98.38%
	100:0	98.98%	99.83%	99.28%	98.38%

Notes on Tables III and IV: Accuracy reflects overall correctness; Precision penalizes false alarms; Recall emphasizes missed attacks; and F1 balances both. Consistently high F1 across splits indicates robustness to class ratio shifts, aligning with the ROC observations in Figs. 6 and 7, and the confusion patterns in Figs. 8 and 9.

REFERENCES

- [1] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, “Edge-iiotset: A new comprehensive realistic cyber security dataset of iiot and iiot applications for centralized and federated learning,” *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022.
- [2] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, “CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment,” *Sensor*, 2023, submitted to Journal of Sensors.

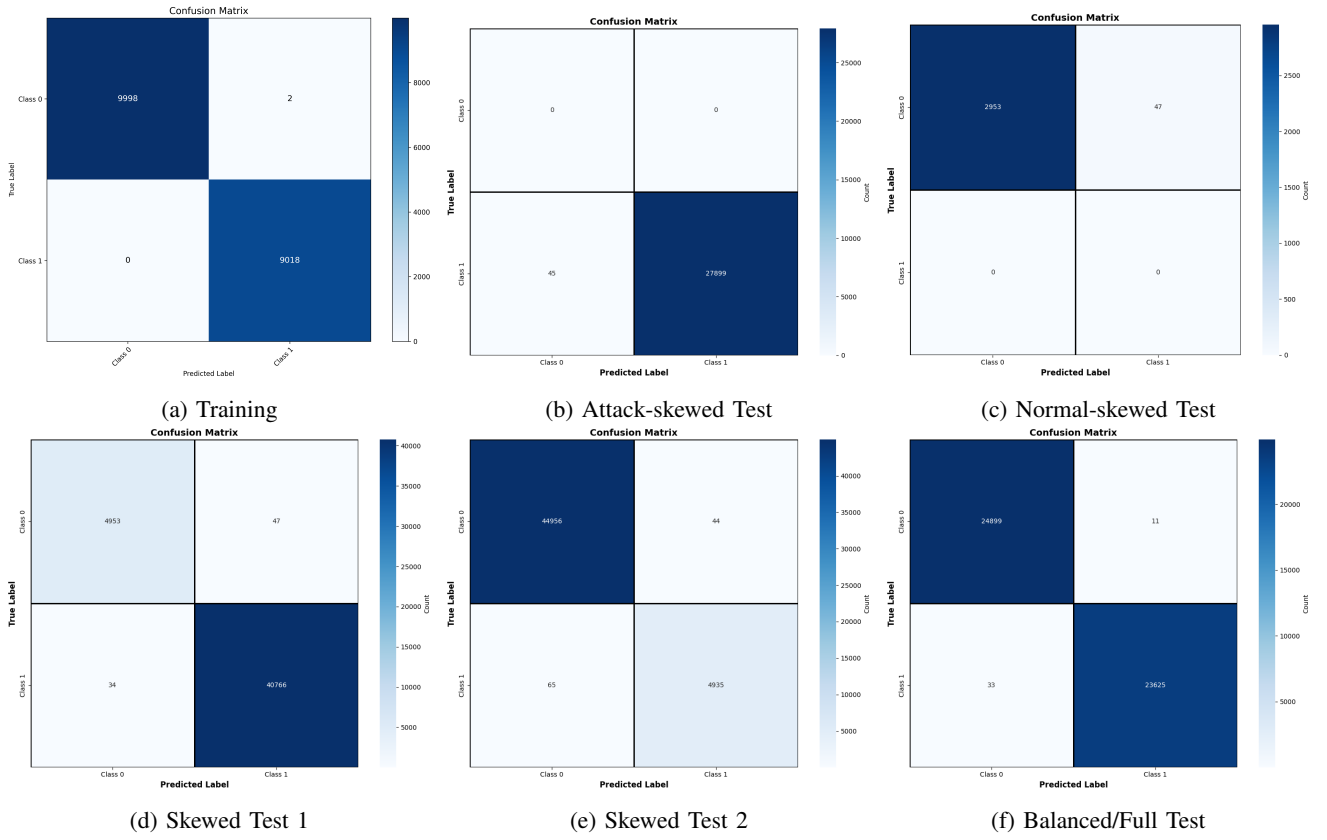


Fig. 8: Clustered confusion matrices for **Edge-IIoT** across training and multiple testing splits.

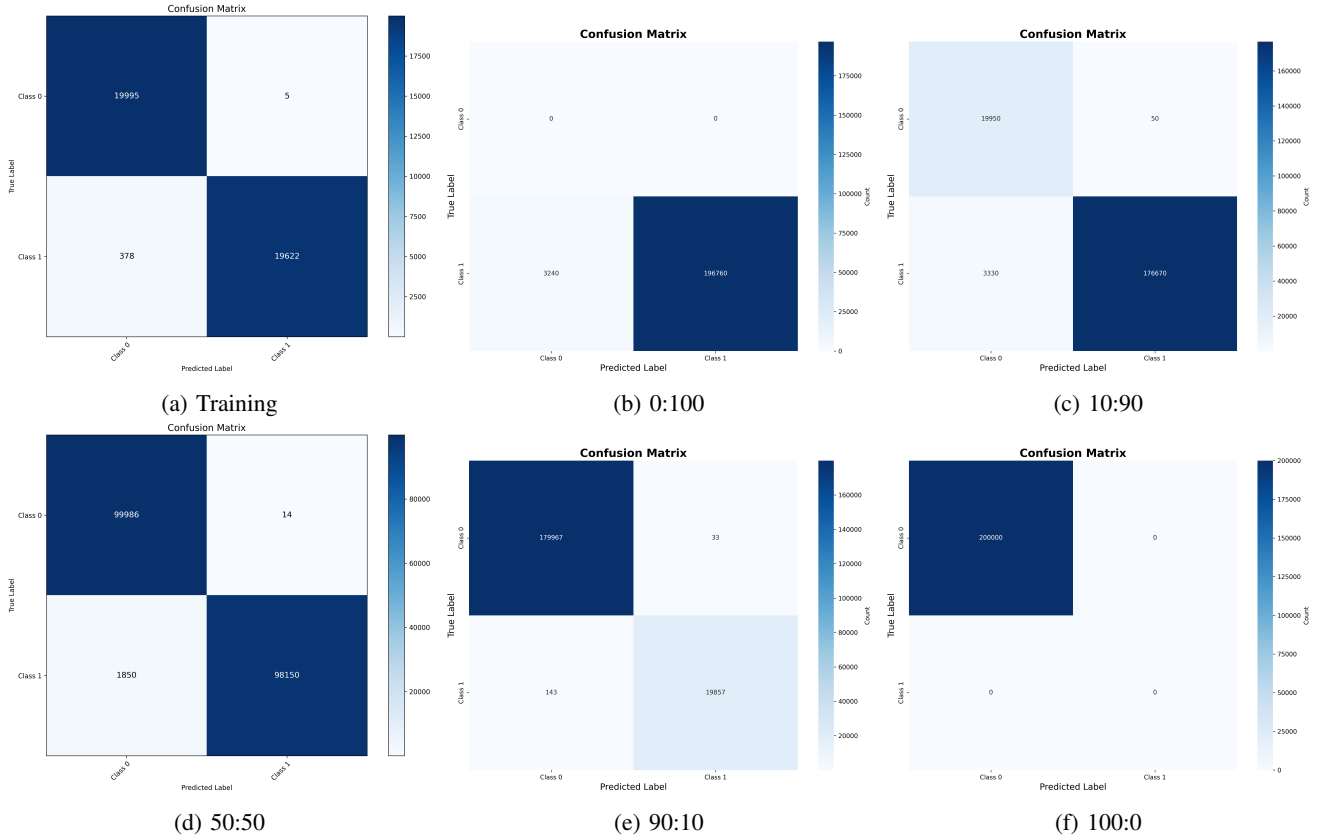


Fig. 9: Clustered confusion matrices for **CICIoT** across training and test splits with Normal:Attack ratios.

- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf